Lab 1

New Attempt

- Due Mar 13 by 11:59pm
- Points 10
- Submitting a file upload

Lab 1: Introduction to Arm TrustZone

In this lab assignment, we will install OP-TEE (Open Portable TEE), which is a Trusted Execution Environment (TEE) that uses Arm TrustZone technology.

Required Operating System: *Ubuntu 20.04, 22.04, or 24.04*

1. Install OP-TEE in Simulation Mode

1. Open the terminal, and use the following command to install the required tools to build OP-TEE:

```
$ apt update && apt upgrade -y
$ apt install -y \
 adb \
 acpica-tools \
 autoconf \
 automake \
 bc \
 bison \
 build-essential \
 ccache \
 cpio \
 cscope \
 curl \
 device-tree-compiler \
 e2tools \
 expect \
 fastboot \
 flex \
 ftp-upload \
 gdisk \
 git \
 libattr1-dev \
 libcap-ng-dev \
 libfdt-dev \
 libftdi-dev \
 libglib2.0-dev \
 libgmp3-dev \
 libhidapi-dev \
 libmpc-dev \
 libncurses5-dev \
 libpixman-1-dev \
 libslirp-dev \
 libssl-dev \
 libtool \
 libusb-1.0-0-dev \
 make \
 mtools \
 netcat \
 ninja-build \
 python3-cryptography \
```

```
python3-pip \
python3-pyelftools \
python3-serial \
python-is-python3 \
rsync \
swig \
unzip \
uuid-dev \
wget \
xalan \
xdg-utils \
xterm \
xz-utils \
zlib1g-dev
```

2. Create a repository called `optee-qemu` to hold OP-TEE, and install the Android repo using `repo init` and `repo sync`:

```
$ mkdir optee-qemu
$ cd optee-qemu
$ repo init -u https://github.com/OP-TEE/manifest.git
$ repo sync
```

3. Enter the `build` directory and build and run OP-TEE. Two UART consoles should then appear on the screen: one for normal world and one for secure world. Note that this step may take a few minutes.

```
$ cd build
$ make toolchains -j2
$ make run
```

4. Note that this step may take a few minutes. To speed up the process, use the command `make -j32 run` (but may not show all error messages). If the `make run` command does not work, check that your `$PATH` environment variable does not contain spaces or other invalid characters, or try using `sudo make run`.

2. Run the `xtest` test suite in simulation mode

1. Once the two consoles appear and the QEMU console stops waiting for input, continue by entering in the QEMU console (the main terminal) the following command:

```
(qemu) c
```

2. In the normal world console, type in `root` or `test`, to login, then use the following command to run the `xtest` test suite, which is the sanity test suite and will perform:

```
# xtest
```

   This will start running many tests that confirm the security features of the TEE. In the normal world monitor, you will be able to see the name of the test being run, and on the secure world monitor you will be able to see the lower level system calls being run.
3. Alternatively, you can run xtest in various other configurations. For more information, please visit the xtest documentation here:

https://optee.readthedocs.io/en/latest/building/gits/optee_test.html#run-xtest (https://optee.readthedocs.io/en/latest/building/gits/optee_test.html#run-xtest)

Submission: **Please save all the terminal output for this task to the file "Lab1_xtest.txt" and submit.** You may need to use the QEMU terminal options to save the output.

3. Run Example Trusted Applications

Now we will build a Trusted Application (TA) using the *TA-devkit*. For more information about running a TA and the structure of a TA, please view the documentation here: https://optee.readthedocs.io/en/latest/building/trusted_applications.html (https://optee.readthedocs.io/en/latest/building/trusted_applications.html)

1. You may notice that in the `optee-qemu/optee-examples` are directories such as `acipher`, `aes`, `hello_world`, etc. You may test these example applications by running `optee_example_[TAName]` in the normal world terminal. For example:
   ```
   # optee_example_hello_world
   ```
2. You should see that along with displaying the "Hello World" message in the secure world terminal, there are also many other system calls are being invoked.

Submission:

(1) Please save all the terminal output for this task to the file "Lab1_TA.txt" and submit.

(2) Please create a report file "Lab1_report.txt" and answer the following questions: What is the first function being called when the Trusted Application is executed? From this function, which parameters come from the normal world, and which parameters come from the secure world?

4a. (Optional Bonus) Implement a Trusted Application

In this section we will develop a digital wallet using OP-TEE. Please implement the digital wallet according to the following specifications:

1. The digital wallet has an initial balance of "1000" securely stored within the TA.
2. It uses a "transactions" data structure within the enclave to log every deposit and payment. Each entry in this log includes the type of action ("deposit" or "pay") and the amount involved. For example: Deposit: 200; Pay: 50.
3. The digital wallet can support four main functions: **deposit**, **pay**, **view balance**, and **view transactions**. Each function should be implemented in `digital_wallet_ta.c`, and have their function ID's located in `digital_wallet_ta.h`:
4. **Deposit**: Increases the current balance by the input amount and logs this deposit in the "transactions" structure.

5. **Pay**: Decreases the current balance by the input amount and logs this payment in the "transactions" structure.
6. **View balance**: Calls the function in the TA to display the current balance on the console.
7. **Check transaction**: Calls the function in TA to display the most recent 3 transactions. If "transactions" contains fewer than three entries, display all of them.
8. Note: your TA file must still a minimum contain the mandatory entry points, which are the `TA_CreateEntryPoint()`, `TA_DestroyEntryPoint()`, `TA_OpenSessionEntryPoint()`, `TA_CloseSessionEntryPoint()`, and `TA_InvokeCommandEntryPoint()` functions.
9. In your untrusted application file (`main.c`), please call all four functions to test their functionality. For reference, the TA file should have the following layout (inside the `optee_examples` directory). Note that it is recommended to start with the `opteehello_world` directory:

```
digital_wallet/
├── ...
└── host
 ├── main.c                    Non-secure application file
└── ta
 ├── Makefile                  BINARY=<uuid>
 ├── Android.mk                Android way to invoke the Makefile
 ├── sub.mk                    srcs-y += digital_wallet_ta.c
 ├── include
 │   └── digital_wallet_ta.h      Header exported to non-secure: TA commands API
 ├── digital_wallet_ta.c          Implementation of TA entry points
 └── user_ta_header_defines.h  TA_UUID, TA_FLAGS, TA_DATA/STACK_SIZE, ...
```

For more information about building Trusted Applications, please visit the documentation here: **https://optee.readthedocs.io/en/latest/building/trusted_applications.html#build-trusted-applications (https://optee.readthedocs.io/en/latest/building/trusted_applications.html#build-trusted-applications)**

Submission:
(1) Please create and submit a zip file of "digital_wallet/" (the outermost directory);
(2) Please save all the terminal output to the file "Lab1_Bonus_DigitalWallet.txt" and submit.

4b. (Optional Bonus) Run OP-TEE in hardware mode

In this bonus part of the lab, you will be running OP-TEE in hardware mode. For this implementation, you will need a Raspberry Pi 3, SD card (it is recommended to be at least 32 GB), and a UART cable, which can be provided by the instructor.

1. Begin by following the similar steps as in part 1 of this lab, building the OP-TEE on your personal computer, **not on the Raspberry Pi**, but stop right before flashing the device. For more information, please visit the document here: **https://optee.readthedocs.io/en/latest/building/gits/build.html#get-and-build-the-solution (https://optee.readthedocs.io/en/latest/building/gits/build.html#get-and-build-the-solution)** , but here is an overview of the steps:

1. Install the same prerequisites as in Part 1 Step 1. Additionally you may find it helpful to build from the following Dockerfile, to ensure that all prerequisites are met: https://optee.readthedocs.io/en/latest/building/prerequisites.html#prerequisites (https://optee.readthedocs.io/en/latest/building/prerequisites.html#prerequisites)
2. Create a repository for the project and install the Android Repo:

```
$ mkdir optee-rpi3
$ cd optee-rpi3
$ repo init -u https://github.com/OP-TEE/manifest.git rpi3.xml
$ repo sync
```

3. Get the toolchains for the project

```
$ cd optee-rpi3/build
$ make -j2 toolchains
```

4. Build the solution

```
$ make -j `nproc`
```

2. At this point, insert your SD card into your computer, and run the following command. You should see the name of your SD card that appears as `/dev/sdx1` or `/dev/sdx2`. Follow the directions that now appear as a result, which will likely involve using the `dd` command to format the SD card. **Be careful at this step to specify the correct drive, or it may wipe out an unwanted drive.**

```
$ make img-help
```

3. Remove the SD card from your personal computer and insert it into the Raspberry Pi. Attach the UART cable as appears below, with the black cable to GND (pin 6), the white cable to UART TX (pin 8), and the green cable to UART RX (pin 10). **Do not attach the red cable**. (Please refer to the following image as a reference for wiring: https://www.jeffgeerling.com/sites/default/files/images/raspberry-pi-serial-cable-connection.png (https://www.jeffgeerling.com/sites/default/files/images/raspberry-pi-serial-cable-connection.png))
4. Attach the USB of the UART cable to your personal computer, and start the monitor using.

```
$ picocom -b 115200 /dev/ttyUSB0
```

5. If picocom is not already installed, use the following command to install it:

```
$ sudo apt-get install picocom
```

6. Power up the Raspberry Pi, and you should see a similar result running on the monitor as it was in simulation mode. When you have a shell, simply run the following command to start the xtest test suite:

```
$ xtest
```

Submission:

(1) Please save all the terminal output to the file "Lab1_Bonus_Hardware.txt" and submit;

(2) Please answer the following question: What are the differences between running OP-TEE in simulation mode and in hardware mode?

References

OP-TEE Documentation: **https://optee.readthedocs.io/en/latest/index.html (https://optee.readthedocs.io/en/latest/index.html)** .

Acknowledgement

This lab assignment is contributed by Jeremy Hui (jwh139@scarletmail.rutgers.edu) based on his course project when taking this course in a previous year. Jeremy will be glad to assist you with the technical questions you may have when working on this lab. Please feel free to reach out to him by email if you need help.